Secure File Storage (AES-256)

Project Documentation Report | Cyber Security | Elevate Labs Internship | 2025

Project Title	Secure File Storage (AES-256)
Developer	Nithish Nidumolu
Domain	Cyber Security
Type	Security Tool — Python + Flask Web Application
Frontend	HTML, CSS, Jinja2 Templates
Backend	Python 3, Flask Framework
Libraries	PyCryptodome, cryptography, os, hashlib
Deployment	Replit Cloud Platform
Live URL	https://aes256-storage--nithishnidumolu.replit.app
Internship	Elevate Labs — Cyber Security Domain
Year	2025

DESCRIPTION

The Secure File Storage (AES-256) is a Python-based security tool designed to encrypt and decrypt files using the AES-256-GCM algorithm — one of the strongest symmetric encryption standards available. It targets sensitive files of any type and protects them with a password-derived 256-bit key, ensuring that only authorised users who possess the correct passphrase can access the original content.

A Flask-based web interface hosted live on Replit allows users to upload any file, provide an encryption password, and download the AES-256 encrypted output directly from the browser. The tool also supports decryption: users upload the encrypted .enc file along with the original password and retrieve the restored plaintext file. Upon every operation, the system generates a structured activity log listing the file name, operation type, timestamp, and status. This project was developed as part of the Elevate Labs Cyber Security Internship.

SYSTEM REQUIREMENTS :

To ensure smooth development, deployment, and usage of the Secure File Storage tool, certain system prerequisites must be met. These requirements are categorised into software, setup, and hardware specifications, all of which contribute to building a robust and deployable security tool.

1. Software Requirements

These are the essential tools and platforms required to develop, test, and run the application efficiently.

- **Operating System:** Windows 10/11, macOS, or Linux — Supports cross-platform development and testing.
- **Python (v3.8 or above):** Core language for the encryption engine and Flask web server. □ Download Python: https://python.org/

- **pip (v21 or above):** Python package manager required to install Flask, PyCryptodome, and cryptography.
- **Flask:** A lightweight Python web framework for building the web interface and handling file upload/download routing.
- **PyCryptodome:** Provides AES-256-GCM encryption and decryption — the core cryptographic engine of the project.
- **cryptography:** Python library offering an alternative AES implementation with Fernet and low-level hazmat primitives.
- **Browser:** Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.
- **Git & GitHub:** For version control and submitting the project to the repository.
- **Replit:** Cloud platform used to host and deploy the Flask application publicly online.
- **Postman:** Optional tool for testing file upload/download API endpoints during development.

2. Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

- **Processor:** Intel Core i3 (7th Gen or above) / AMD Ryzen 3 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 4 GB (8 GB recommended) — For handling the development server, IDE, and browser testing simultaneously.
- **Storage:** At least 500 MB free space — Required for Python, pip package installations, and project files.
- **Display:** 1366×768 or higher — Recommended for optimal coding experience and application layout visualisation.
- **Internet:** Stable broadband connection — Required for deploying on Replit and uploading/downloading files during testing.

3. Setup Commands

Install all required Python libraries using pip:

```
pip install flask pycryptodome cryptography
```

Run the Flask web application locally:

```
python app.py
```

The application will start on: http://localhost:5000

SCENARIO-BASED CASE STUDY

Scenario: Riya, a junior data analyst at a healthcare company, needs to securely send a patient records CSV to a colleague over email. Before attaching it, her IT security team asks her to encrypt the file using the Secure File Storage tool to comply with data protection policies.

1. Accessing the Tool :

➤Riya opens her browser and navigates to https://aes256-storage--nithishnidumolu.replit.app
➤She sees the clean interface with a file upload widget, a password field, and an Encrypt File button.
➤She selects the patient_records.csv file from her local drive.

2. Encrypting the File :

➤Riya enters a strong password: P@ssw0rd#Secure2025 and clicks Encrypt File.
➤The Flask backend receives the file and password via a POST request to the /encrypt route.
➤The tool derives a 256-bit AES key from the password using SHA-256, generates a random 16-byte IV, and encrypts the file using AES-256-GCM.
➤A download link for patient_records.csv.enc appears on screen.

3. Downloading the Encrypted File :

➤Riya clicks the download link and saves patient_records.csv.enc to her desktop.
➤She attaches the .enc file to the email — even if intercepted, the contents are unreadable without the password.
➤The tool also logs the operation: File: patient_records.csv | Operation: ENCRYPT | Status: SUCCESS.

4. Decrypting the File :

➤Her colleague receives the .enc file, opens the tool, uploads patient_records.csv.enc, and enters the shared password.
➤The Flask backend calls the decrypt route, verifies the AES-GCM authentication tag, and decrypts the ciphertext.
➤The original patient_records.csv is restored and offered as a download.

5. Viewing the Activity Log :

➤Riya sees the activity log rendered on the web page with timestamps for each operation.
➤Each entry shows the file name, operation type (ENCRYPT / DECRYPT), timestamp, and status (SUCCESS / FAILED).
➤The tool also saves a structured log to file_storage_log.txt for audit purposes.

6. Taking Action :

➤Riya shares the file_storage_log.txt with her IT security team as proof of compliant data handling.
➤The team confirms that AES-256-GCM with a random IV meets their data-at-rest and data-in-transit encryption policy.
➤A re-test with a wrong password confirms decryption correctly fails — returning an Authentication tag mismatch error.

PROJECT ARCHITECTURE

Technical Architecture

The Secure File Storage tool follows a client-server architecture where the Flask framework serves both the frontend interface and the backend encryption logic. The frontend is a simple HTML/CSS page rendered via Jinja2 templates. The backend is a Python-Flask application that handles incoming file upload requests, runs the AES-256-GCM encryption or decryption engine, and streams the processed file back to the browser.

The frontend incorporates the Jinja2 templating engine to dynamically render operation results and activity log entries. Flask handles all routing and communication between the user interface and the encryption engine. For data storage, activity logs are saved to a local file_storage_log.txt file with timestamps, operation types, and status codes.

Together, the frontend templates, Flask routing layer, encryption engine (crypto.py), and file-based log system form a comprehensive technical architecture for this security tool. This architecture enables accurate AES-256 encryption and decryption, efficient file handling, and seamless deployment on the Replit cloud platform.

Architecture Table

Layer	Technology	Responsibility
Presentation	HTML, CSS, Jinja2	User interface — file upload form, download links, status display
Application	Python, Flask	Route handling, encryption orchestration, key management
Encryption Engine	PyCryptodome / cryptography	AES-256-GCM encrypt / decrypt, IV generation, tag verification
Key Layer	hashlib, os.urandom	Derives 256-bit key from password via SHA-256, generates random IV
Storage Layer	Python file I/O	Saves .enc files with prepended IV+tag; reads them back for decryption
Deployment	Replit Cloud	Public hosting accessible via web browser

FEATURES :

1. AES-256-GCM File Encryption

- **FILE ENCRYPTION:** The tool automatically encrypts any uploaded file using AES-256-GCM — a high-security authenticated encryption scheme that provides both confidentiality and data integrity.
- **RANDOM IV GENERATION:** A unique 16-byte random Initialisation Vector (IV) is generated for every encryption operation, ensuring that encrypting the same file twice always produces different ciphertext.

2. AES-256-GCM File Decryption

- **TAG VERIFICATION:** AES-GCM authentication tag verification is performed before any decryption — if the file has been tampered with or the password is wrong, decryption is rejected with a clear error message.
- **FILE RESTORATION:** On successful verification, the original plaintext file is fully restored and offered as a browser download.

3. Password-Based Key Derivation

- **KEY DERIVATION:** Uses SHA-256 hashing to derive a deterministic 256-bit symmetric key from the user-supplied password, ensuring the same password always produces the same key.
- **KEY SAFETY:** The derived key is never stored on disk — it exists only in memory during the encrypt/decrypt operation.

4. Flask Web Interface

- **FILE UPLOAD FORM:** Clean web interface where users upload any file, provide a password, and trigger encryption or decryption with a single button click.

- **RESULTS DISPLAY:** Operation results are rendered dynamically on the results page with colour-coded status badges (SUCCESS in green, FAILED in red).

5. Activity Log Generation

- **AUTO-SAVED LOG:** Every encrypt and decrypt operation automatically appends an entry to file_storage_log.txt with the file name, operation type, timestamp, and status.
- **STATUS RATINGS:** Each log entry is categorised as SUCCESS or FAILED for easy auditing and compliance reporting.

ROLES AND RESPONSIBILITIES :

1. Security Intern / Developer

Example: Riya, a data analyst who needs to secure sensitive files before transmission, opens the Secure File Storage tool. She uploads the target file, provides a password, and uses the Encrypt File button to protect it. Once the encrypted .enc file is downloaded, she sends it securely. The activity log confirms a successful operation.

2. Running the Initial Encryption

Example: Riya navigates to https://aes256-storage--nithishnidumolu.replit.app, uploads patient_records.csv, enters her password, and clicks Encrypt File. The tool derives the AES-256 key, generates a random IV, encrypts the file using GCM mode, and returns the patient_records.csv.enc download.

3. Reviewing Encryption Output

Example: The tool confirms successful encryption with a SUCCESS badge, the original file size, and the .enc output file size. Riya also sees the IV length (16 bytes) and GCM authentication tag length (16 bytes) logged in the activity entry.

4. Reviewing Decryption Results

Example: Her colleague uploads patient_records.csv.enc and enters the shared password. The tool verifies the GCM authentication tag, decrypts the ciphertext, and returns the original patient_records.csv file — identical byte-for-byte to the original.

5. Exporting and Sharing the Log

Example: Riya opens file_storage_log.txt and shares it with her IT compliance team. The structured log lists every operation with file name, type, timestamp, and status — ready for audit review.

6. Error Verification

Example: After testing, Riya tries to decrypt the .enc file using a wrong password. The tool correctly rejects the operation — returning Authentication tag mismatch — and logs the attempt as FAILED. No partial decryption occurs.

7. Deployment and Submission

Example: Riya submits the project to her GitHub repository with the encryption tool code, README, and log. The live Replit deployment is shared with Elevate Labs as proof of a working, deployed security tool.

USER FLOW :

1. Security User / Developer :

- **Role:** Use the tool to encrypt sensitive files before storage or transmission, and decrypt them when access is needed.

Flow:

1. Accessing the Tool:

- Open the browser and navigate to https://aes256-storage--nithishnidumolu.replit.app
- The home page loads with a file upload widget, password field, and Encrypt / Decrypt buttons.

2. Encrypting a File:

- Upload the target file using the file picker.
- Enter a strong password in the password field.
- Click Encrypt File — the backend derives the AES-256 key and runs GCM encryption.

3. Viewing Results:

- Results page shows the operation type, status badge, original filename, and .enc output filename.
- Colour-coded status badges make it easy to confirm success or diagnose failures.

4. Downloading the Encrypted File:

- The encrypted .enc file is available as a browser download immediately after encryption.
- Share the .enc file with the recipient along with the password through a separate secure channel.

2. Recipient / Developer (Decryption) :

- **Role:** Receive the .enc file and decrypt it to recover the original plaintext file.

Flow:

1. Upload Encrypted File:

- Open the tool and upload the received .enc file using the file picker.

2. Enter Password:

- Type the shared password in the password field and click Decrypt File.

3. Verify and Download:

- The tool verifies the GCM tag and decrypts the file — original file is returned as a download.

4. Check Log:

- Review file_storage_log.txt to confirm the decryption operation was logged with SUCCESS status.

ENCRYPTION ARCHITECTURE PATTERN :

The Secure File Storage backend follows a modular pattern separating the web layer (Flask routes), the logic layer (crypto functions), and the output layer (log generation). This separation allows for modularity, easier maintenance, and scalability.

Flask Routes Layer (Web Layer)

The routes layer is responsible for handling all HTTP file upload and download requests. It receives the uploaded file and password from the user, delegates processing to the encryption engine, and returns the processed file to the browser via a download response.

Encryption Engine Layer (Logic Layer)

The encryption engine (crypto.py) acts as the core logic layer. It receives the file bytes and password, derives the AES-256 key, generates a random IV, performs AES-GCM encryption or decryption, and returns the result. All cryptographic logic lives here.

Log / Output Layer

The log generator appends a structured entry to file_storage_log.txt after every operation. It is decoupled from the encryption logic, making it easy to replace with a database or SIEM integration in future versions.

Advantages of This Architecture:

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability.
- **Scalability:** New encryption modes (e.g., RSA hybrid encryption) can be added easily by extending crypto.py.
- **Reusability:** The encryption functions can be reused via CLI or API without the Flask layer.
- **Testability:** Each layer can be tested independently — routes, crypto logic, and log output.
- **Portability:** The tool can be deployed on any cloud platform (Replit, Heroku, AWS) with minimal changes.

PROJECT SETUP AND CONFIGURATION :

Creating the project folder

1. Create a new folder named secure-file-storage.
2. Inside that folder create two files: app.py and crypto.py.
3. Create a templates/ folder with index.html and result.html.
4. Create a static/ folder with style.css.
5. Open the folder in VS Code.

Python environment setup

- Open the project folder in the VS Code terminal.
- Create a virtual environment (optional but recommended):

```
python -m venv venv
```

- Activate the virtual environment:

```
venv\Scripts\activate    # Windows
source venv/bin/activate  # macOS / Linux
```

- Install all dependencies:

```
pip install flask pycryptodome cryptography
```

Flask app setup

- Create app.py and import Flask.
- Define routes: / for the home page, /encrypt for file encryption, and /decrypt for file decryption.
- Set host='0.0.0.0' and port=5000 for Replit compatibility.
- Run the development server:

```
python app.py
```

BACKEND DEVELOPMENT :

https://aes256-storage--nithishnidumolu.replit.app

Set Up Project Structure:

- Create app.py as the main Flask entry point and crypto.py as the core encryption engine.
- Install necessary dependencies: Flask, PyCryptodome, and cryptography.

Create Flask Web Server:

- Set up a Flask server to handle HTTP requests and serve the web interface.
- Configure the home route (/), encrypt route (/encrypt), and decrypt route (/decrypt) with appropriate HTTP methods.

Define Encryption Routes:

- Create the /encrypt POST route to receive the uploaded file and password from the user.
- Implement the route handler to call encrypt_file() and return the .enc file as a Flask send_file() response.
- Create the /decrypt POST route to receive the .enc file and password and call decrypt_file().

Implement Encryption Functions:

- Build derive_key(password) to hash the password with SHA-256 and return a 32-byte AES key.
- Build encrypt_file(file_bytes, password) to generate a random 16-byte IV, encrypt using AES-256-GCM, and return IV + tag + ciphertext.
- Implement decrypt_file(enc_bytes, password) to extract the IV and tag, verify the GCM authentication tag, and return the plaintext bytes.
- Implement get_file_details(file) to extract original filename, size, and MIME type for the results page.

Log Generation:

- Implement append_log(filename, operation, status) to append a structured entry to file_storage_log.txt.

- Include timestamp, original filename, operation type (ENCRYPT / DECRYPT), and status (SUCCESS / FAILED) in each log entry.

Error Handling:

- Wrap all crypto operations in try/except blocks to handle incorrect passwords and corrupted files.
- Return a clear FAILED status and error message when the GCM authentication tag does not match.

DATA STORAGE :

Unlike database-driven applications, the Secure File Storage tool uses a simple file-based logging system. Every encrypt and decrypt operation automatically appends a new entry to file_storage_log.txt with structured details.

Log File Structure:

- **Original Filename:** The original name of the file that was processed.
- **Timestamp:** Date and time the operation was performed.
- **Operation Type:** ENCRYPT or DECRYPT.
- **Status:** SUCCESS if the operation completed correctly; FAILED if authentication or decryption failed.
- **File Size:** The size of the input file in bytes.
- **Output Filename:** The name of the output file (.enc for encryption, original name for decryption).

Sample Log Output:

```
============================================================
SECURE FILE STORAGE — ACTIVITY LOG
============================================================
Timestamp   : 2025-03-23 14:35:12
File        : patient_records.csv
Operation   : ENCRYPT
Status      : SUCCESS
Input Size  : 48,320 bytes
Output File : patient_records.csv.enc
------------------------------------------------------------
Timestamp   : 2025-03-23 14:41:07
File        : patient_records.csv.enc
Operation   : DECRYPT
Status      : SUCCESS
Input Size  : 48,352 bytes
Output File : patient_records.csv
------------------------------------------------------------
Timestamp   : 2025-03-23 14:45:30
File        : patient_records.csv.enc
Operation   : DECRYPT
Status      : FAILED — Authentication tag mismatch
```

FRONTEND DEVELOPMENT :

https://aes256-storage--nithishnidumolu.replit.app

1. Setup Flask Templates:

Building the Secure File Storage frontend involves a straightforward development process. The foundation is built using Flask's Jinja2 templating system. This includes creating the template structure, adding HTML/CSS styling, and organising the project files.

- Create the templates/ directory.
- Add index.html (home page with encrypt/decrypt forms) and result.html (operation result and log display page).

2. Design UI Components:

Reusable HTML components are built for all interactive elements — the file upload widget, the password input field, the Encrypt / Decrypt buttons, the results table, and the status badges. A dark security-themed stylesheet (style.css) defines the overall look and feel. Navigation between the home and result pages is handled by Flask routing.

3. Implement Frontend Logic:

In the final stage of frontend development, the HTML forms are connected to Flask routes via POST requests using standard HTML file upload (multipart/form-data). This includes:

- Connecting the file upload form to the /encrypt and /decrypt POST routes.
- Rendering results dynamically using Jinja2 {{ }} template variables for filename, status, and log entries.

PROJECT EXECUTION.

Steps for Project Execution:

Step 1: Clone or Download the Project:

- **a)** Clone from GitHub or open directly in Replit.
- **b)** Ensure all files are present: app.py, crypto.py, templates/, static/

Step 2: Install Dependencies:

- **a)** Open a terminal and navigate to the project folder.
- **b)** Install all required packages:

```
pip install flask pycryptodome cryptography
```

Step 3: Configure Environment:

- **a)** Ensure Python 3.8+ is installed.
- **b)** Verify Flask is installed by running:

```
flask --version
```

Step 4: Start the Flask Server:

- Inside the project folder, run:

```
python app.py
```

http://localhost:5000

OUTPUT SCREENSHOTS :

- **Home Page / Landing Page :** Displays the tool title and description with a file upload widget, password input field, and Encrypt File / Decrypt File buttons. Serves as the entry point for all users.
- **Encryption In Progress :** After clicking Encrypt File, the page shows a processing state while the backend derives the AES key, generates the IV, and encrypts the uploaded file.
- **Results Page — Encryption Successful :** Displays the encryption result with a green SUCCESS badge, original filename, .enc output filename, file size before and after, and a download link for the encrypted file.
- **Results Page — Decryption Successful :** Displays the decryption result with a green SUCCESS badge, restored original filename, and a download link for the decrypted file.
- **Results Page — Decryption Failed :** If the wrong password is provided, the page shows a red FAILED badge with the message: Authentication tag mismatch — incorrect password or corrupted file.
- **Generated Log (file_storage_log.txt) :** A structured text log appended automatically after every operation. Contains timestamp, file name, operation type, status, file sizes, and output filename.
- **Admin / Code View on Replit :** Shows the Replit IDE with the project files (app.py, crypto.py, templates/) and the live deployment panel confirming the app is running publicly.